My Playbook: Guide + Member A

UPI Payment Failure Diagnosis, Reconciliation & Fraud Risk System — what I do, step by step

I hold two roles. As Member A I build the payment core (backend). As Guide I review and sign off the whole project. The one rule that keeps this honest: I never approve my own work. Member B or C reviews every Member A pull request, and the Guide sign-off at each checkpoint is based on the checks below, not on my own opinion of my code.

Part 1 — Role 1: Guide

The Guide does not write feature code. The job is to keep three tracks fitting together, catch problems early, and make sure the team can defend the project in the viva.

1.1 Phase checkpoints I must sign off

Phase	I approve only when I can see this
0. Design lock	Schema (9 tables), API contract (JSON shapes) and state machine diagram are written in docs/ and each of B and C can explain them without looking.
1. Skeleton	Each track runs standalone: backend starts and creates tables, fraud checks run on sample input, both UIs open with mock data.
2. Happy path	A payment sent from the UPI app reaches the backend, gets a fraud decision, and the UI shows it. Live demo works once.
3. Real contribution	A debited-but-not-credited payment is caught automatically and shown with a reason and recovery estimate. Duplicate key creates exactly one transaction. Threaded vs sequential benchmark numbers exist.
4. Hardening	All 10 scenarios in docs/test-scenarios.md are marked pass with screenshots. Logs trace one transaction end to end.
5. Polish	I personally run the full demo from a clean clone, following the written script, with no help.

1.2 Pull request review checklist

- [ ] Does it stay inside its own track's folder? If not, why?
- [ ] Does it change the API contract or schema? If yes, did all 3 members agree?
- [ ] Is the idempotency check a DB unique constraint, not just an if-statement?
- [ ] Is ML kept out of the diagnosis engine, and rules kept out of the fraud model?
- [ ] Is there a test or a recorded manual run showing it works?
- [ ] Is the PR small enough to review in 15 minutes?

1.3 Weekly rhythm

When	What	Output
Start of week (15 min)	Each member states: done last week, doing this week, blocked on.	Blockers list
Mid-week	Review open PRs; resolve any cross-track disagreement.	Merged PRs, decisions noted in docs/
End of week	Merge dev into main only if the phase checkpoint passes.	Tagged release, e.g. phase-2-done

1.4 Risks I watch for

- **Contract drift:** a track changes a field name and breaks the others. Fix: contract lives in docs/ and changes need 3 approvals.
- **Scope creep:** ML creeping into the diagnosis engine, or extra features before the core works. Fix: refuse until the phase checkpoint passes.
- **SQLite threading errors** once threads start. Fix: check_same_thread=False and one session per thread.
- **No benchmark numbers** for sequential vs concurrent. This is the viva proof, so it must exist before Phase 5.
- **Live demo failure.** Fix: simulators have a switch to force each failure on demand, plus seeded demo data.

1.5 Viva preparation

By Phase 5 every member must answer these without notes. I run a mock viva with them:

Question	Where the answer lives
What exactly is the problem and why does it happen?	Fault-explanation document (distributed transaction, CAP)
What does UPI already do, and what is your gap?	Project summary, section 3
How do you decide the failure reason? Do you need history?	Diagnosis decision table; rules, no data needed
Where is ML used and why only there?	Fraud model + recovery time only; related-work PDF
Which dataset?	Synthetic fraud data + simulator logs; limitation stated
How do you prevent duplicate payments?	DB unique constraint on idempotency_key
What happens when money is debited but not credited?	State machine -> RECONCILIATION_REQUIRED -> alerts
Prove concurrency helps.	Sequential vs ThreadPoolExecutor benchmark table

Part 2 — Role 2: Member A (Backend / Payment Core)

I own the ground truth: database, payment APIs, state machine, idempotency and the fake banks. Members B and C code against what I build, so contract accuracy matters more than speed.

2.1 Phase 0 — Design lock (days 1–2)

[ ] Draft the 9-table schema: customers, accounts, merchants, devices, transactions, transaction_events, risk_results, alerts, fraud_cases. Put it in docs/schema.md.

- [ ] Write the exact JSON for POST /transactions (request and response) and GET /payment-status/{id}.
- [ ] Draw the state machine (table in 2.4) and put it in docs/architecture.md.
- [ ] Send all three to B and C for review. Do not start coding until both reply.

2.2 Phase 1 — Core skeleton

Step 1. Project setup. Create a virtual environment inside backend/, install fastapi, uvicorn, sqlalchemy, pydantic. Layout: app/core/db.py, app/models/, app/api/, app/simulators/, app/main.py.

Step 2. Database. SQLite engine with check_same_thread=False, a SessionLocal factory, and a get_db dependency. Call create_all on startup.

Step 3. Models. Implement the 9 tables. Key constraints: transactions.idempotency_key UNIQUE and NOT NULL; transaction_events has no update or delete code anywhere; risk_results stores thread_name, started_at, finished_at (Member B fills these).

Step 4. Bank simulators. Two plain functions: sender_bank.debit(account_id, amount) and receiver_bank.credit(account_id, amount). Each returns {status, code, latency_ms}. A shared config decides the mode:

Mode	Behaviour	Used to demo
SUCCESS	Responds OK after a short delay	Normal payment
TIMEOUT	Sleeps longer than the timeout, returns nothing	Debited but receiver silent
ERROR	Returns an error code	Credit failed / invalid receiver
SLOW	Succeeds but after a long delay	Late credit, status check later finds it
LOST_ACK	Credit applied but confirmation never returned	Case 4: credited, confirmation lost

Add POST /sim/config so the mode can be switched live during the demo without restarting.

Step 5. First endpoint. POST /transactions only creates the row and returns an ID. Seed script creates 2 customers, their accounts, one merchant and one device.

Definition of done: server starts, tables exist, a POST returns an ID, simulators can be called directly and respect their mode.

2.3 Phase 2 — Happy path

- [ ] Wire POST /transactions: validate, create row (INITIATED), call fraud hook, call debit, call credit, set final state.
- [ ] Fraud hook first returns a stub like (APPROVE, 0). Swap in Member B's real function when ready; agree the function signature in Phase 0.
- [ ] GET /transactions/{id} and GET /payment-status/{id} return real data including the event timeline.
- [ ] Validate inputs: positive amount, sender is not receiver, accounts exist. Return clear error messages.

2.4 Phase 3 — State machine and idempotency (the core contribution)

State machine. Keep one dictionary of allowed transitions and one function, transition(txn, new_state), that checks it. Every state change must go through that function and write one transaction_events row.

From	Allowed next states
INITIATED	PROCESSING, FAILED

From	Allowed next states
PROCESSING	DEBITED, FAILED, TIMEOUT
DEBITED	CREDIT_PENDING
CREDIT_PENDING	CREDITED, TIMEOUT, RECONCILIATION_REQUIRED
CREDITED	COMPLETED
TIMEOUT	CREDITED (status check finds it), RECONCILIATION_REQUIRED, FAILED
RECONCILIATION_REQUIRED	COMPLETED (resolved), FAILED (reversed to sender)
COMPLETED, FAILED	None (final)

```
ALLOWED = {
  'INITIATED': {'PROCESSING', 'FAILED'},
  'PROCESSING': {'DEBITED', 'FAILED', 'TIMEOUT'},
  'DEBITED': {'CREDIT_PENDING'},
  # ... rest from the table above
}

def transition(db, txn, new_state, detail=''):
    if new_state not in ALLOWED.get(txn.state, set()):
        raise ValueError(f'Illegal {txn.state} -> {new_state}')
    db.add(TransactionEvent(transaction_id=txn.id, from_state=txn.state,
                            to_state=new_state, detail=detail))
    txn.state = new_state
    db.commit()
```

Idempotency. The client sends an idempotency_key. Insert the row and let the database enforce uniqueness. If the insert raises IntegrityError, roll back, load the existing transaction, and return it. Never use a plain 'check then insert', because two simultaneous requests can both pass the check. If the same key arrives with a different amount or receiver, return 409 instead of silently reusing it.

Debit-check and credit-check. After a debit succeeds, start a credit watcher with a timeout (for example 10 seconds in the demo). If no credit confirmation arrives, move CREDIT_PENDING to TIMEOUT, run one status check against the receiver simulator, and if the credit is still absent move to RECONCILIATION_REQUIRED and raise an alert. Each side's check runs as its own task so they verify independently.

Definition of done: the same key sent twice, even concurrently, creates exactly one transaction; an illegal transition raises an error; every transaction has a complete event trail.

2.5 Phase 4 — Hardening

- [ ] Add GET /alerts, GET /cases, GET /logs/{id} for Member C's dashboard.
- [ ] Check every endpoint against the contract: field names, types, status codes.
- [ ] Write a concurrency test script: fire the same idempotency_key from 20 threads, assert one row exists.
- [ ] Force each simulator mode and confirm the final state and events match docs/test-scenarios.md.
- [ ] Add structured logging with the transaction id on every line.

2.6 My endpoints at a glance

Endpoint	Purpose	Phase
POST /transactions	Create payment, run flow, return decision and state	1–3
GET /transactions/{id}	Transaction details	2
GET /payment-status/{id}	Current state plus event timeline	2
GET /risk/{id}	Fraud result (data from Member B)	2
GET /alerts, /cases, /logs/{id}	Feeds the ops dashboard	4
POST /sim/config	Switch bank simulator mode live	1

Part 3 — Doing both roles without cutting corners

Risk of holding both roles	How I handle it
I approve my own code	Member B or C reviews every Member A PR. I never merge my own PR.
I skip sign-off checks because I know the code	Checkpoint criteria in 1.1 are written down; I tick them from observed behaviour, not memory.
Guide work gets squeezed by building	Fixed slots: 15 min start of week, 30 min mid-week reviews, 1 hour end-of-week checkpoint.
I become the bottleneck for B and C	Finish and publish the contract and simulators first; unblocking others comes before my own features.

Order of work that unblocks everyone fastest: (1) schema and contract, (2) simulators with the mode switch, (3) POST /transactions returning real data, then (4) idempotency and the state machine.